

30 Java String Coding Problems with Solutions

30 Java String Coding Problems with Solutions

1. Reverse a String
 2. Check if a String is Palindrome
 3. Count Occurrences of a Character
 4. Find Duplicate Characters in a String
 5. Remove Duplicates from a String
 6. Check if Two Strings are Anagrams
 7. Find First Non-Repeated Character
 8. Check if a String Contains Only Digits
 9. Count Vowels and Consonants
 10. Check if a String is a Rotation of Another String
 11. Find the longest word in a sentence
 12. Convert a string to Title Case
 13. Find all permutations of a string
 14. Convert string to integer without Integer.parseInt()
 15. Find the frequency of words in a sentence
 16. Convert String to char[] without .toCharArray()
 17. Remove all non-alphabetic characters from a string
 18. Convert lowercase to uppercase without toUpperCase()
 19. Reverse words in a sentence
 20. Check if two strings are one edit distance apart
 21. Implement strStr() function (Find substring index)
 22. Compress a string (aaabb -> a3b2)
 23. Find longest common prefix in an array of strings
 24. Implement isSubstring() without contains()
 25. Print all substrings of a string
 26. Convert a number to words (123 -> "One Hundred Twenty-Three")
 27. Validate an email address format
 28. Convert CamelCase to snake_case
 29. Find the lexicographically smallest string
 30. Remove consecutive duplicate words from a sentence
-

1. Reverse a String

```
public class ReverseString {  
    public static String reverse(String str) {  
        return new StringBuilder(str).reverse().toString();  
    }  
  
    public static void main(String[] args) {  
        System.out.println(reverse("hello")); // Output: olleh  
    }  
}
```

2. Check if a String is Palindrome

```
public class PalindromeCheck {  
    public static boolean isPalindrome(String str) {  
        return str.equals(new StringBuilder(str).reverse().toString());  
    }  
  
    public static void main(String[] args) {  
        System.out.println(isPalindrome("madam")); // true  
    }  
}
```

3. Count Occurrences of a Character

```
public class CharCount {  
    public static int countChar(String str, char ch) {  
        return (int) str.chars().filter(c -> c == ch).count();  
    }  
  
    public static void main(String[] args) {  
        System.out.println(countChar("hello", 'l')); // 2  
    }  
}
```

4. Find Duplicate Characters in a String

```
import java.util.*;  
  
public class DuplicateCharacters {  
    public static void findDuplicates(String str) {  
        Map<Character, Integer> map = new HashMap<>();  
        for (char ch : str.toCharArray()) {  
            map.put(ch, map.getOrDefault(ch, 0) + 1);  
        }  
        map.forEach((k, v) -> {  
            if (v > 1) System.out.println(k + ": " + v);  
        });  
    }  
  
    public static void main(String[] args) {  
        findDuplicates("programming");  
    }  
}
```

5. Remove Duplicates from a String

```
1 public class RemoveDuplicates {
2     public static String removeDuplicates(String str) {
3         return str.chars().distinct()
4             .collect(StringBuilder::new, StringBuilder
5                 ::appendCodePoint, StringBuilder::append)
6             .toString();
7     }
8     public static void main(String[] args) {
9         System.out.println(removeDuplicates("helloooaa")); // helo
10    }
11 }
12
```

6. Check if Two Strings are Anagrams

```
import java.util.Arrays;

public class AnagramCheck {
    public static boolean isAnagram(String s1, String s2) {
        char[] a1 = s1.toCharArray();
        char[] a2 = s2.toCharArray();
        Arrays.sort(a1);
        Arrays.sort(a2);
        return Arrays.equals(a1, a2);
    }

    public static void main(String[] args) {
        System.out.println(isAnagram("listen", "silent")); // true
    }
}
```

7. Find First Non-Repeated Character

```
import java.util.LinkedHashMap;
import java.util.Map;

public class FirstNonRepeated {
    public static char firstNonRepeated(String str) {
        Map<Character, Integer> map = new LinkedHashMap<>();
        for (char ch : str.toCharArray()) {
            map.put(ch, map.getOrDefault(ch, 0) + 1);
        }
        for (Map.Entry<Character, Integer> entry : map.entrySet()) {
            if (entry.getValue() == 1) return entry.getKey();
        }
        return '_';
    }

    public static void main(String[] args) {
        System.out.println(firstNonRepeated("swiss")); // w
    }
}
```

8. Check if a String Contains Only Digits

```
public class OnlyDigits {
    public static boolean isNumeric(String str) {
        return str.matches("\\d+");
    }

    public static void main(String[] args) {
        System.out.println(isNumeric("12345")); // true
        System.out.println(isNumeric("12a45")); // false
    }
}
```

9. Count Vowels and Consonants

```
1 public class CountVowelsConsonants {
2     public static void countVowelsConsonants(String str) {
3         int vowels = 0, consonants = 0;
4         str = str.toLowerCase();
5         for (char ch : str.toCharArray()) {
6             if ("aeiou".indexOf(ch) != -1) vowels++;
7             else if (Character.isLetter(ch)) consonants++;
8         }
9         System.out.println("Vowels: " + vowels + ", Consonants: "
10            + consonants);
11     }
12     public static void main(String[] args) {
13         countVowelsConsonants("Hello");
14     }
15 }
16
```

10. Check if a String is a Rotation of Another String

```
public class StringRotation {
    public static boolean isRotation(String s1, String s2) {
        return (s1.length() == s2.length()) && (s1 + s1).contains(s2);
    }

    public static void main(String[] args) {
        System.out.println(isRotation("abcd", "cdab")); // true
    }
}
```

11. Find the longest word in a sentence

```
1 import java.util.*;
2 public class StringProblems {
3
4     // 11. Find the longest word in a sentence
5     public static String longestWord(String sentence) {
6         return Arrays.stream(sentence.split(" "))
7             .max(Comparator.comparingInt(String::length))
8             .orElse("");
9     }
10    public static void main(String[] args) {
11        System.out.println("Longest Word: " + longestWord("Java is
12            a powerful programming language"));
13    }
14 }
```

12. Convert a string to Title Case

```
1 import java.util.*;
2 public class StringProblems {
3
4     public static String toTitleCase(String str) {
5         String[] words = str.split(" ");
6         StringBuilder sb = new StringBuilder();
7         for (String word : words) {
8             sb.append(Character.toUpperCase(word.charAt(0)))
9                 .append(word.substring(1).toLowerCase())
10                .append(" ");
11        }
12        return sb.toString().trim();
13    }
14    public static void main(String[] args) {
15        System.out.println("Title Case: " + toTitleCase("hello
16            world from java"));
17    }
18 }
```

13. Find all permutations of a string

```
1 import java.util.*;
2 public class StringProblems {
3
4     public static void permute(String str, String result) {
5         if (str.isEmpty()) {
6             System.out.println(result);
7             return;
8         }
9         for (int i = 0; i < str.length(); i++) {
10             permute(str.substring(0, i) + str.substring(i + 1),
11                     result + str.charAt(i));
12         }
13
14     public static void main(String[] args) {
15         System.out.println("Permutations of 'abc':");
16         permute("abc", "");
17     }
18 }
```

14. Convert string to integer without Integer.parseInt()

```
1 import java.util.*;
2 public class StringProblems {
3
4     public static int stringToInt(String str) {
5         int num = 0;
6         for (char c : str.toCharArray()) {
7             num = num * 10 + (c - '0');
8         }
9         return num;
10    }
11    public static void main(String[] args) {
12        System.out.println("String to Integer: " + stringToInt
13                             ("1234"));
14    }
15 }
```


15. Find the frequency of words in a sentence

```
1 import java.util.*;
2 public class StringProblems {
3
4     public static void wordFrequency(String sentence) {
5         Map<String, Integer> map = new HashMap<>();
6         for (String word : sentence.split(" ")) {
7             map.put(word, map.getOrDefault(word, 0) + 1);
8         }
9         map.forEach((k, v) -> System.out.println(k + ": " + v));
10    }
11
12    public static void main(String[] args) {
13        System.out.println("Word Frequency in 'java java python c++
14        c++ java:");
15        wordFrequency("java java python c++ c++ java");
16    }
```

16. Convert String to char[] without .toCharArray()

```
1 import java.util.*;
2 public class StringProblems {
3
4     // 16. Convert String to char[] without .toCharArray()
5     public static char[] stringToCharArray(String str) {
6         char[] chars = new char[str.length()];
7         for (int i = 0; i < str.length(); i++) {
8             chars[i] = str.charAt(i);
9         }
10        return chars;
11    }
12
13    public static void main(String[] args) {
14        System.out.println("Convert String to char[]: " + Arrays
15        .toString(stringToCharArray("hello")));
16    }
```

17. Remove all non-alphabetic characters from a string

```
1 import java.util.*;
2 public class StringProblems {
3
4     // 17. Remove all non-alphabetic characters from a string
5     public static String removeNonAlphabets(String str) {
6         return str.replaceAll("[^a-zA-Z]", "");
7     }
8
9     public static void main(String[] args) {
10         System.out.println("Remove Non-Alphabets: " +
11             removeNonAlphabets("h3ll0 w@rld!"));
12     }
13 }
```

18. Convert lowercase to uppercase without toUpperCase()

```
1 import java.util.*;
2 public class StringProblems {
3
4     // 18. Convert lowercase to uppercase without toUpperCase()
5     public static String toUpperCaseManual(String str) {
6         StringBuilder sb = new StringBuilder();
7         for (char c : str.toCharArray()) {
8             sb.append((c >= 'a' && c <= 'z') ? (char) (c - 32) : c);
9         }
10        return sb.toString();
11    }
12
13    public static void main(String[] args) {
14        System.out.println("To UpperCase: " +
15            toUpperCaseManual("hello"));
16    }
17 }
```

19. Reverse words in a sentence

```
1 import java.util.*;
2 public class StringProblems {
3
4     // 19. Reverse words in a sentence
5     public static String reverseWords(String sentence) {
6         String[] words = sentence.split(" ");
7         Collections.reverse(Arrays.asList(words));
8         return String.join(" ", words);
9     }
10
11     public static void main(String[] args) {
12         System.out.println("Reverse Words: " + reverseWords
13             ("hello world from java"));
14     }
15 }
```

20. Check if two strings are one edit distance apart

```
1 import java.util.*;
2 public class StringProblems {
3
4     // 20. Check if two strings are one edit distance apart
5     public static boolean isOneEditDistance(String s1, String s2)
6     {
7         if (Math.abs(s1.length() - s2.length()) > 1) return false;
8         int count = 0, i = 0, j = 0;
9         while (i < s1.length() && j < s2.length()) {
10             if (s1.charAt(i) != s2.charAt(j)) {
11                 if (++count > 1) return false;
12                 if (s1.length() > s2.length()) i++;
13                 else if (s1.length() < s2.length()) j++;
14                 else { i++; j++; }
15             } else { i++; j++; }
16         }
17         return true;
18     }
19     public static void main(String[] args) {
20         System.out.println("One Edit Distance: " +
21             isOneEditDistance("cat", "cut"));
22     }
```

21. Implement strStr() function (Find substring index)

```
1 import java.util.*;
2 public class StringProblems {
3
4     // 21. Implement strStr() function (Find substring index)
5     public static int strStr(String haystack, String needle) {
6         return haystack.indexOf(needle);
7     }
8
9     public static void main(String[] args) {
10         System.out.println("strStr: " + strStr("hello", "ll"
11             ));
12     }
```

22. Compress a string (aaabb -> a3b2)

```
1 import java.util.*;
2 public class StringProblems {
3
4     // 22. Compress a string (aaabb -> a3b2)
5     public static String compressString(String str) {
6         StringBuilder sb = new StringBuilder();
7         int count = 1;
8         for (int i = 1; i <= str.length(); i++) {
9             if(i==str.length() || str.charAt(i) != str.charAt(i-1)) {
10                 sb.append(str.charAt(i - 1)).append(count);
11                 count = 1;
12             } else {
13                 count++;
14             }
15         }
16         return sb.toString();
17     }
18
19
20     public static void main(String[] args) {
21         System.out.println("Compressed: " + compressString
22             ("aaabb"));
23     }
```

23. Find longest common prefix in an array of strings

```
1 import java.util.*;
2 public class StringProblems {
3
4     // 23. Find longest common prefix in an array of strings
5     public static String longestCommonPrefix(String[] strs) {
6         if (strs.length == 0) return "";
7         String prefix = strs[0];
8         for (String str : strs) {
9             while (!str.startsWith(prefix)) {
10                 prefix = prefix.substring(0, prefix.length() - 1);
11                 if (prefix.isEmpty()) return "";
12             }
13         }
14         return prefix;
15     }
16
17     public static void main(String[] args) {
18         System.out.println("Longest Prefix: " +
19             longestCommonPrefix(new String[]{"flower", "flow",
20                 "flight"}));
21     }
22 }
```

24. Implement isSubstring() without contains()

```
1 import java.util.*;
2 public class StringProblems {
3
4     // 24. Implement isSubstring() without contains()
5     public static boolean isSubstring(String str, String sub) {
6         return str.indexOf(sub) != -1;
7     }
8     public static void main(String[] args) {
9         System.out.println("isSubstring: " + isSubstring("hello
10             world", "world"));
11     }
12 }
```

25. Print all substrings of a string

```
1 import java.util.*;
2 public class StringProblems {
3
4     // 25. Print all substrings of a string
5     public static void printSubstrings(String str) {
6         for (int i = 0; i < str.length(); i++) {
7             for (int j = i + 1; j <= str.length(); j++) {
8                 System.out.println(str.substring(i, j));
9             }
10        }
11    }
12    public static void main(String[] args) {
13        printSubstrings("abc");
14    }
15 }
```

26. Convert a number to words (123 -> "One Hundred Twenty-Three")

```
1 import java.util.*;
2 public class StringProblems {
3
4     // 26. Convert a number to words (123 -> "One Hundred Twenty
      -Three")
5     public static String numberToWords(int num) {
6         String[] belowTen = {"", "One", "Two", "Three", "Four",
          "Five", "Six", "Seven", "Eight", "Nine"};
7         String[] belowTwenty = {"Ten", "Eleven", "Twelve",
          "Thirteen", "Fourteen", "Fifteen", "Sixteen",
          "Seventeen", "Eighteen", "Nineteen"};
8         String[] belowHundred = {"", "", "Twenty", "Thirty",
          "Forty", "Fifty", "Sixty", "Seventy", "Eighty",
          "Ninety"};
9         if (num < 10) return belowTen[num];
10        if (num < 20) return belowTwenty[num - 10];
11        if (num < 100) return belowHundred[num / 10] + " " +
          belowTen[num % 10];
12        return belowHundred[num / 100] + " Hundred " + numberToWords
          (num % 100);
13    }
14    public static void main(String[] args) {
15        System.out.println("Number to Words: " + numberToWords(100
          ));
16    }
17 }
```


27. Validate an email address format

```
1 import java.util.*;
2 public class StringProblems {
3
4     // 27. Validate an email address format
5     public static boolean isValidEmail(String email) {
6         return email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$");
7     }
8     public static void main(String[] args) {
9         System.out.println("Valid Email: " + isValidEmail
10                             ("test@example.com"));
11     }
12 }
```

28. Convert CamelCase to snake_case

```
1 import java.util.*;
2 public class StringProblems {
3
4     // 28. Convert CamelCase to snake_case
5     public static String camelToSnake(String str) {
6         return str.replaceAll("([a-z])([A-Z])", "$1_$2")
7             .toLowerCase();
8     }
9     public static void main(String[] args) {
10         System.out.println("Camel to Snake: " + camelToSnake
11                             ("camelCaseString"));
12     }
13 }
```

29. Find the lexicographically smallest string

```
1 import java.util.*;
2 public class StringProblems {
3
4     // 29. Find the lexicographically smallest string
5     public static String smallestLexicographic(String[] arr) {
6         Arrays.sort(arr);
7         return arr[0];
8     }
9     public static void main(String[] args) {
10         System.out.println("Smallest Lexicographic: " +
11             smallestLexicographic(new String[]{"banana", "apple",
12                 "cherry"}));
11     }
12 }
```

30. Remove consecutive duplicate words from a sentence

```
1 import java.util.*;
2 public class StringProblems {
3
4     // 30. Remove consecutive duplicate words from a sentence
5     public static String removeConsecutiveDuplicates(String
        sentence) {
6         String[] words = sentence.split(" ");
7         StringBuilder sb = new StringBuilder();
8         String prev = "";
9         for (String word : words) {
10             if (!word.equals(prev)) {
11                 sb.append(word).append(" ");
12                 prev = word;
13             }
14         }
15         return sb.toString().trim();
16     }
17     public static void main(String[] args) {
18         System.out.println("Removed Duplicates: " +
            removeConsecutiveDuplicates("hello hello world world
            java java"));
19     }
20 }
```

Happy learning! 🌸